FinVantage - Product Requirements Document

Nov 7, 2026 · @Ashzaeen

FinVantage is a responsive web app plus a Chrome extension that simulates a user's financial future and steps in at the moment of a money decision, instead of showing a dashboard after the fact.

1. Overview

Vision. An AI advisor that looks forward, explains itself, and speaks up where decisions actually happen (a rental listing, a checkout page, a car quote), grounded in the user's whole financial picture: checking, savings, CDs, IRAs, loans, credit cards, and mortgages.

What changed from the idea doc (v1.0):

- Section 6.1 (Monitoring Agent) is replaced by the **Omnipresent Copilot**, an in-the-wild contextual advisor delivered as a Chrome extension.
- Section 5.2 (Lifestyle Creep Score) and 5.3 (Tax-Aware Nudges) are out of scope, along with the v2 Opportunity Scanner (6.4), which depended on 5.3.
- Everything else stands: table-stakes features, Life-Event Simulator, Advisory Agent, Quarterly Report Agent, hybrid life-event entry, data model, and cost approach.

Goals

1. Show a user their complete financial picture across all six account types in one view.
2. Simulate their future as a range of outcomes, not one number.
3. Intercept high-impact purchases and show the effect on cash flow, goals, and debt before the user commits.
4. Explain every number and recommendation in plain language.
5. Be a strong resume and portfolio showcase, and a competitive HackUTD entry (PNC 360 Financial Health Navigator track, design track, and the sponsor tracks in section 3.1).

Non-goals (v1): auto-executing anything (trades, payments, cancellations), multi-person or household coordination, estate planning, real production bank connections, tax advice, creep scoring.

2. Problem, Users, and Differentiation

Problem. People hold money in many places (checking, savings, CDs, IRAs, auto and personal loans, credit cards, mortgages) yet few have one clear view of how they are doing or what to do next. Existing apps (Mint, YNAB, Copilot, Rocket Money, Credit Karma) converged on linking accounts, categorizing, budgeting, and net worth. They share two gaps: they look backward, and they are passive. Users only see consequences after the money is spent.

Target user. Anyone managing their own money, with no niche targeting. Two reference personas drive design decisions:

- **The starter** (student or early-career): few accounts, one or two loans, no budget habit. Needs plain-language explanations and a safe spending threshold.
- **The planner** (a few years into a career): several account types, saving toward a house or a car. Needs trade-off analysis and a probability range on goals.

Differentiation

Typical finance app	FinVantage
Shows what happened last month	Simulates where finances are headed, as a range of outcomes
Lives inside its own app, opened occasionally	Copilot appears on Zillow, checkout, and car-financing pages at the point of commitment
Goals tracked one at a time	Goals simulated together, competing for the same dollars
Charts the user must interpret	Plain-language explanation of every number and recommendation
Passive	Proactive, but never acts without the user saying yes

3. Scope and Priorities

The full vision is larger than a 24-hour hackathon build, so every feature carries a tier. P0 is the demo-critical slice, P1 is built if time allows, P2 is post-hackathon.

Feature	In scope	Tier
Synthetic multi-account data (6 account types)	Yes	P0
Account and net worth views	Yes	P0
Life-Event Simulator (Monte Carlo, fan chart, what-if sliders)	Yes	P0
Omnipresent Copilot extension (rent listing and checkout scenarios)	Yes	P0
Savings goals and recurring expense detection	Yes	P1
Budget envelopes, monthly spending summary, bill reminders	Yes	P1
Advisory Agent (ranked options with trade-offs)	Yes	P1
Free-text life-event parser	Yes	P1
Car-financing Copilot scenario	Yes	P1
Investment portfolio view, credit score tracking (mocked)	Yes	P2
Quarterly Report Agent	Yes	P2
Real bank linking through Plaid	Yes (sandbox only)	P2
Lifestyle Creep Score (old 5.2)	No	Removed
Tax-Aware Nudges (old 5.3) and Opportunity Scanner (old 6.4)	No	Removed
Old Monitoring Agent (old 6.1)	No	Replaced by Copilot

Out of scope for v1: auto-executing actions, multi-person or household modeling, estate planning, production bank connections, native mobile app.

3.1 Hackathon Track Strategy

The goal is to enter every track where a sponsor technology has a real job in the product. Judges for "best use of" tracks reward genuine use shown in the demo and the submission, not a logo on a slide, so each entry below names that job. Sponsor integrations should take about four hours in total; PNC and design quality come first.

Track	Fit	The job it does in FinVantage	Effort	Decision
PNC 360 Financial Health Navigator	Direct	The whole product: six account types, simulator, Copilot	Core	Enter
Design track	Good	Fan chart, what-if sliders, Copilot overlay card	Shared	Enter
Tiger Data	Strong	Relational core on Tiger Data Postgres: accounts, transactions, time-bucketed spending summaries. Replaces Supabase	Low	Enter
MongoDB Atlas	Good	Document store for simulation snapshots, Copilot events, life events, and agent memory. These are nested, variable-shape records	Low to medium	Enter
Vultr	Good	Hosts the API and simulator worker on a Vultr instance. The extension needs a public HTTPS endpoint, so this is a real requirement	Low, about 1 hour	Enter

Track	Fit	The job it does in FinVantage	Effort	Decision
GoDaddy Registry	Trivial	A memorable domain for the app and API; the slide shows code MLHUTD84	Minutes	Enter
SpaceXSI (Cursor and Grok Bot)	Process only	Build with Cursor and say so in the submission; check the sponsor's rules for what Grok Bot expects	Low	Enter if rules are light
Spacetime	Possible	Live push of Copilot events and projection updates to the dashboard	High, new paradigm	Stretch, only with a dedicated owner
Solana	Weak	No honest role; a forced crypto angle would weaken the PNC pitch	None	Skip
8090.AI	Stretch	Legacy Data Adapter agent that translates old bank record formats into our schema (see below the table)	Medium, about 3 hours	Stretch, only if the P0 slice is done
Best Use of Gemini API (Google Cloud)	Strong	All LLM work: Copilot card, Advisory Agent, life-event parsing, legacy file mapping	Low, replaces Claude	Enter

Notes

- Two data stores have distinct jobs: Tiger Data for relational ledger data, MongoDB Atlas for documents. Do not add Spacetime as a third store unless one person owns it, since it also shifts server logic into the database.

- Submission rules and prize eligibility were read from slides only. Confirm each sponsor's requirements (links on the slides, or the sponsor tables) before relying on them.
- Every sponsor track needs one visible moment in the demo or the submission write-up, such as the architecture slide naming each sponsor's role.

Legacy Data Adapter (stretch, for 8090.AI). The track asks for agents and tools that make old software easier to understand, improve, or replace. A pitch of "we modernize how legacy systems handle finances" alone would read as a bolt-on, so the entry needs a working tool behind it:

1. The user or demo drops in a legacy bank export: a fixed-width mainframe-style flat file with a COBOL-style copybook describing its layout, or an older statement format such as BAI2.
2. An agent infers the field layout, maps each field into FinVantage's Account and Transaction schema, and lists any record it cannot map.
3. The UI shows a before-and-after diff, and the normalized accounts appear in the net worth view and feed the Copilot.

Pitch line: "Banks still run on decades-old record formats. FinVantage's adapter agent translates them into data a modern advisor can reason over."

Scope: P1 stretch, about 3 hours, built only on synthetic sample files we write ourselves. We do not claim it works on any real bank's systems. Acceptance: the sample file produces normalized accounts in the app, and unmappable rows are listed.

4. Table-Stakes Features

These ten features make the app feel like a real product. All use sandbox or synthetic data. The synthetic profile must include all six PNC account types: checking, savings, CDs and IRAs, auto and personal loans, credit cards, and mortgages.

#	Feature	Requirement	Acceptance criteria	Tier
1	Account linking	Connect accounts via Plaid sandbox, or load a synthetic profile JSON	A demo user shows all six account types with balances	P0

#	Feature	Requirement	Acceptance criteria	Tier
2	Auto-categorized transactions	Rule-based categorization first, LLM fallback for unknown merchants	90% of synthetic transactions categorized without user input	P1
3	Budget envelopes	Monthly limit per category with progress bar	Overspend state is visible and updates on new transactions	P1
4	Net worth tracking	Assets minus liabilities across all accounts, over time	One chart and a breakdown by account type	P0
5	Recurring expense detection	Detect subscriptions and repeat charges from transaction history	Recurring charges listed with cadence and next date	P1
6	Investment portfolio view	Holdings, allocation, benchmark comparison	Allocation chart and benchmark line render from mock data	P2
7	Bill reminders	Alerts before due dates, based on detected recurring charges	Upcoming bills list for the next 14 days	P1
8	Savings goals	Target amount and date with visual progress	Goal cards show progress and feed the simulator	P1
9	Credit score tracking	Provider-pulled or mocked score with history	Mock score and trend shown, labeled as mocked	P2
10	Monthly spending summary	Recap of where the month's money went	Summary by category with change versus prior month	P1

5. Feature: Life-Event Simulator

The simulator runs thousands of randomized scenarios and returns a range of outcomes per goal instead of one falsely precise number. It is the math engine behind both the dashboard and the Copilot.

Functional requirements

1. **Inputs.** Current balances across all accounts, income and spending history, goals (target, date, priority, flexibility flag), and logged life events (type, date, rough dollar impact).
2. **Randomization.** Each run samples income changes, spending variation, and market returns. Default 5,000 paths; configurable.
3. **Joint simulation.** All goals are simulated together, competing for the same dollars by priority order, not as independent calculations.
4. **Output.** For each goal, a probability range (for example "60-85% likely") and a fan chart of balances over time (median plus percentile band).
5. **What-if controls.** Sliders for monthly savings, a purchase amount, and event timing, with the result updating in under 2 seconds for the demo.
6. **Snapshots.** Every run stores its inputs and outputs, and links to the previous run so any two runs can be diffed.
7. **Callable API.** The Copilot calls the simulator with a hypothetical purchase and receives the change in goal probability and goal date.

Non-functional

- Deterministic when given a seed, so demos and tests are repeatable.
- Runs server-side in TypeScript or Python; heavy runs are batched, not blocking the UI.

Acceptance criteria: logging a house purchase event visibly lowers the probability range on a linked savings goal, and the fan chart widens over time.

6. Feature: Omnipresent Copilot (replaces Monitoring Agent)

The Copilot is a Chrome extension (Manifest V3) that detects a high-impact financial decision on the page the user is viewing and shows a slide-in card with the impact on their finances before they commit. Core thesis: a financial AI should intercept decisions at the point of commitment, not wait inside a banking app for a post-mortem.

What it replaces. The old Monitoring Agent re-ran simulations on triggers (large transactions, income changes, life events) and sent digests. In the new design, the simulator is still the engine, but the trigger is the user's own browsing intent. Digest-style monitoring can return later as a P2 add-on.

Functional requirements

1. **Page detection.** A content script identifies the page type and extracts price data using lightweight DOM parsing: rental or home listing, checkout or cart, and (P1) vehicle financing quote. Mock local pages must work for demos, since live sites change their markup.
2. **Context call.** The extension sends a minimal payload (price, item type, merchant) to the backend. No page content beyond that is sent or stored.
3. **Deterministic math core.** The backend computes every number in code before any LLM call: fixed-cost ratio and debt-to-income shift, liquid runway (cash divided by daily burn), goal delay from the simulator, and opportunity cost of the amount invested over a stated horizon.
4. **LLM synthesis.** The model turns the computed numbers into a short card, under 60 words, in three parts: Impact, Trade-off, and Smart Move. The model may not invent or alter figures.
5. **Overlay UI.** A floating badge expands into a card rendered in a Shadow DOM so host-page CSS cannot leak in. The card shows the key numbers, the plain-language advice, and a link to open the full simulation in the web app.
6. **Suggestion only.** The Copilot never moves money, applies for credit, or clicks anything on the host page.
7. **User control.** The user can dismiss a card, snooze it per site, or turn the extension off. Dismissals are respected for the session.

Scenarios

Scenario	Trigger	Math core output	Card advice	Tier
Apartment hunting (Zillow, Apartments.com)	Viewing a $1,950/month listing	Fixed-cost ratio 42% to 68%, goal delay in months, safe rent threshold for the profile	Safe alternative at or under the threshold, with filtered matches if available	P0
Major checkout (Best Buy, Amazon, Apple)	$1,499 laptop in cart	Liquid buffer after purchase versus the user's target, days until next paycheck	Split payment between checking and a 0% promo card with a payoff schedule	P0

Scenario	Trigger	Math core output	Card advice	Tier
Car shopping (CarMax, dealerships)	60-month financing quote on a $28,000 car	Total cost of ownership, DTI before and after	Warns before the user submits a credit application	P1

The dollar figures above are demo values from the concept brief; real values come from the synthetic profile.

Example card (checkout scenario)

> Paying in full drops your checking buffer below your $2,500 target until payday on the 15th. Split it: $600 from checking and the rest on your 0% promo card with $150/month autopay, and you pay zero interest.

Acceptance criteria

- Opening the demo rental listing and the demo checkout page each produce a card within 3 seconds.
- Every figure on a card matches the math core's output exactly.
- Changing the profile data changes the card without any code change.
- No page text other than the extracted price fields leaves the browser.

Edge cases: price not found (show nothing, never guess), multiple prices on a page (use the one in the primary listing or cart total), currency other than USD (skip), user has no linked data (prompt to load the demo profile).

7. Feature: Advisory and Quarterly Report Agents

Both agents stand from the original plan. The Advisory Agent now consumes Copilot and simulator output instead of Monitoring Agent events.

7.1 Advisory Agent (P1)

1. Takes a situation (a Copilot event, a simulator change, or a user question) and returns 2 to 3 ranked options with explicit trade-offs, for example "cut $150/month in dining" versus "push the goal back 8 months".
2. Tailors options to the goal's flexibility flag: non-negotiable goals get spending-cut suggestions, flexible goals get timeline suggestions.

3. Shows its reasoning: each option lists the numbers it rests on and links to the simulation run behind them.
4. Never presents one bossy answer and never acts on the user's behalf.

Acceptance criteria: for a goal whose probability dropped, the agent returns at least two options, each with a quantified effect on that probability.

7.2 Quarterly Report Agent (P2)

1. Writes a quarter-over-quarter recap covering what went well, what did not, and why.
2. Compares to the previous quarter's report, not just a snapshot.
3. Ends with a look ahead and one or two concrete next steps.
4. Source data is the stored simulation runs, Copilot events, and recommendations.
5. Delivered as an in-app text report; a downloadable PDF is a stretch goal.

Acceptance criteria: a generated report cites at least one number from each of two consecutive quarters and names the cause of the largest change.

8. Life-Event Entry (Hybrid)

There are two ways to log a life event, and both produce the same structured record, so the simulator never cares which was used.

Path 1: pick from a list (P0). A predefined set of event types (for example having a child, buying a house, changing jobs, moving), each with a short guided form for date and key dollar amounts. Reliable, fast to build, and guarantees clean data.

Path 2: type it out (P1). The user describes the event in their own words. A lightweight model converts it into the same structured format and shows the result for confirmation before it is saved.

Requirements

1. Both paths write the same Life Event schema (type, date, recurring or one-time, dollar impact, duration).
2. The free-text parser only maps to the categories the form supports in v1; unknown events return "I could not match that" with the list of supported types.
3. Nothing is saved until the user confirms the parsed result. A misread sentence must never silently change a simulation.
4. A saved event triggers a new simulation run and shows the change in goal probability.

Acceptance criteria: the sentence "we're having a baby in March" parses to a child event with a March date and a confirmation screen, and cancelling leaves the data unchanged.

9. Data Model

Seven entities hold everything (the idea doc's six, with Account and Transaction listed separately). Exact fields will be finalized by the team; this is the shape. The Agent Event record from the idea doc becomes a Copilot Event record.

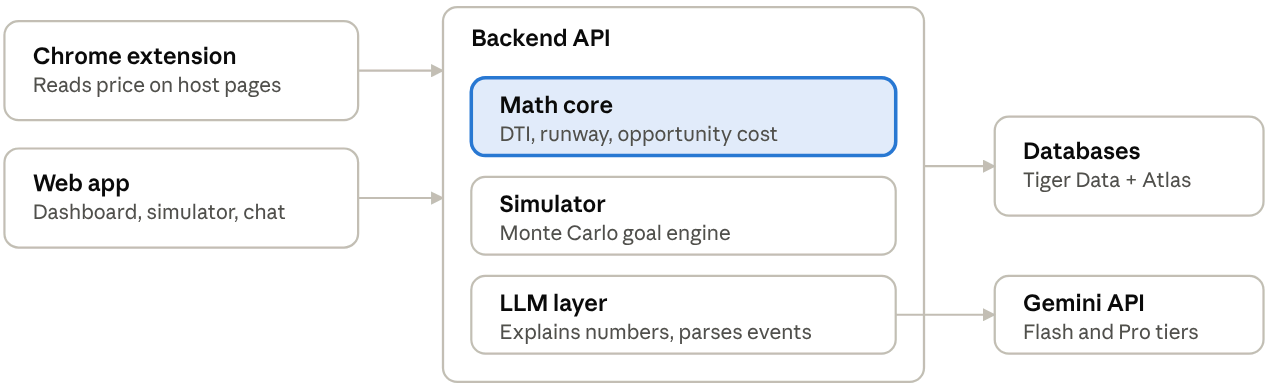
Entity	Purpose	Key fields
User	Profile, preferences, which goals are non-negotiable versus flexible	id, display name, preferences, extension settings
Account	Linked or synthetic accounts of all six types	id, user, type (checking, savings, CD, IRA, loan, credit card, mortgage), balance, rate or APR, minimum payment
Transaction	Stream feeding categorization, budgets, recurring detection, and the simulator	id, account, date, amount, merchant, category, recurring flag
Life Event	Events entered through the hybrid approach	id, user, type, date, dollar impact, duration, source (form or parsed)
Goal	Target amount and date, competing for the same dollars	id, user, target, date, priority rank, flexibility flag
Simulation Run	Saved snapshot so any two runs can be diffed	id, inputs snapshot, priority setup, per-goal outcome range, previous run id, seed
Copilot Event and Recommendation	What the Copilot saw, the computed numbers, and what it advised; feeds the Advisory and Quarterly Report agents	id, user, page type, merchant, extracted price, computed metrics, card text, user action (dismissed, opened, snoozed), timestamp

Design decisions carried over

- Every simulation run stores both inputs and outputs, so comparison reads stored results instead of recalculating.
- Goal priority is saved per run, not as a single fixed value. This lets the system separate "this got worse because of spending" from "this got worse because you reprioritized" by re-running the comparison twice, once holding priorities steady and once holding spending steady. This attribution is P2.
- Goals are simulated together, which costs more compute per comparison. Cap paths and cache results to stay inside the 2-second what-if target.

10. Architecture, Stack, and LLM Use

The math core computes every number before the LLM writes a word



system architecture · 2 clients, 1 backend, 2 services

Both clients call one backend. The highlighted math core produces every figure the user sees; the LLM layer only words them.

Proposed stack

Layer	Choice	Notes
Web app	Next.js, responsive, Tailwind	Desktop and mobile browsers; no native app in v1
Extension	Chrome Manifest V3, content script, Shadow DOM overlay	Mock local pages for demo reliability

Layer	Choice	Notes
Backend	Node.js (TypeScript) or FastAPI	One language for simulator and math core is simpler
Database	Tiger Data Postgres and MongoDB Atlas	Tiger Data holds accounts and transactions; Atlas holds snapshots, events, and agent memory
Bank data	Plaid sandbox or synthetic JSON	No real accounts in v1
Hosting	Vercel free tier for the web app, a Vultr instance for the API and simulator	Free tiers cover build and demo

LLM routing

Task	Model	Reason
Parse a typed life event	Gemini Flash-tier model	Cheap and fast, with structured JSON output
Copilot card, advice, quarterly report	Gemini Pro-tier model	Reasoning and clear writing
Legacy Data Adapter: infer a file layout from a sample file or statement image	Gemini Pro-tier model (multimodal input)	Multimodal input and long context handle odd formats

Decision: the team uses Gemini for all LLM work, to enter the Best Use of Gemini API track. This replaces the Claude models and pricing in the idea doc; use the current model names when building. Gemini features with a real job here: structured JSON output for life-event parsing, function calling so the agents call the math core and simulator as tools, and multimodal input for the Legacy Data Adapter. All model calls go through one wrapper, so swapping a model or provider is a one-file change.

Guardrails

- Math first: all figures computed in code, then passed to the model as fixed inputs. Output is checked so every number in the card appears in the input.
- Card text capped at 60 words, using a fixed Impact, Trade-off, Smart Move structure.

- Context caching for the user's standing financial profile; batch processing for non-urgent jobs such as quarterly reports.

11. Non-Functional Requirements, Safety, and Compliance

Area	Requirement
Performance	Copilot card within 3 seconds of page load; what-if simulation result within 2 seconds
Reliability	If the LLM call fails, the Copilot shows a deterministic template card built from the math core, so the demo never shows an empty state
Privacy	The extension sends only price, item type, and merchant. No browsing history, no page text, no screenshots. Permissions limited to listed host domains
Security	API keys live on the server only. The extension and web client never hold model or bank keys
Data	Sandbox or synthetic data only. Mocked values (credit score, portfolio) are labeled as mocked in the UI
Explainability	Every number links to its inputs; every recommendation lists the figures it rests on
Safety	Suggest only. No money movement, no credit applications, no clicks on host pages, no auto-executed actions
Accessibility	Keyboard-navigable overlay, readable contrast, responsive layouts down to phone width

Disclaimers. FinVantage provides educational projections, not financial advice. Simulator outputs are ranges and estimates, not guarantees, and the UI says so near every probability.

Compliance (out of scope for v1). Connecting real users' real accounts brings compliance and cost considerations (production bank-data agreements, data protection, and financial regulation) that are not addressed here. They must be reviewed before any move

past sandbox data.

12. Success Metrics, Cost, Work Split, and Build Order

12.1 Success metrics

There are no live users in v1, so success is measured on the demo and on engine quality. Targets are proposals for the team to confirm.

Metric	Target
Copilot cards correct on both P0 scenarios	100% of figures match the math core
Time to Copilot card	3 seconds or less
What-if simulation latency	2 seconds or less
Free-text life-event parse, supported categories	Correct on the team's test sentences, confirmed before save
Live demo	Add a transaction or open a listing and watch the projection or card update, without a restart

12.2 Cost to run

Figures come from the idea doc's planning estimates and published pricing at the time of writing; check each vendor's current pricing before committing a budget.

Cost line	Demo build (about 50 users or fewer)	Bigger scale (about 1,000 users)
Hosting	$0 (free tier)	$50-$150
Database	$0 (free tier)	$50-$100
LLM usage (idea doc estimate based on Claude pricing; recheck for Gemini)	$15-$25	$300-$500
Bank data (Plaid)	$0-$100 (sandbox)	$300-$1,000

Cost line	Demo build (about 50 users or fewer)	Bigger scale (about 1,000 users)
Domain and misc.	$1-$2	$1-$2
Rough total per month	**$20-$165**	**$700-$1,750**

For the actual demo build, expect to land near the low end, under $30 a month. Cost levers: route each task to the cheapest capable model, use context caching, stay on sandbox data, and keep agent memory in the existing databases.

12.3 Work split

Chunks are claimable by anyone, based on interest and availability, and can change hands.

Chunk	Tier
Synthetic data and account views (net worth, accounts, goals)	P0
Simulation engine	P0
Copilot extension: page detection and overlay UI	P0
Copilot backend: math core and LLM card generation	P0
Auth, data model, infrastructure	P0
Design and UI polish	P0
Life-event form and free-text parser	P0 form, P1 parser
Advisory Agent prompts and recommendation logic	P1
Transaction pipeline, categorization, budgets, recurring detection	P1
Quarterly report generation	P2

12.4 Suggested build order

1. Synthetic profile with all six account types, plus the math helpers (runway, DTI, opportunity cost).
2. Simulation engine with fan chart and what-if sliders.
3. Extension shell with the rental and checkout mock pages and the overlay card.
4. LLM card generation with the math-first guardrail and the template fallback.
5. Life-event form, then the free-text parser.
6. Advisory Agent, remaining table-stakes views, then polish and the 2-minute demo script.
7. Sponsor integrations (Vultr deploy, domain, Tiger Data and MongoDB wiring) start in the first hours, not the last. Anything left on P2 becomes a roadmap slide.

13. Risks, Open Questions, and Next Steps

Risks

Risk	Impact	Mitigation
Scope is larger than a 24-hour build	Half-finished features, weak demo	Hold to the P0 tier; everything else is a roadmap slide
Host-page DOM changes break detection on live Zillow or Amazon	Copilot shows nothing during the demo	Demo on mock local pages first, live sites as a bonus
LLM invents or alters a figure	User sees a wrong number	Math-first design, output check that numbers match input, template fallback
Simulator too slow with joint goals	What-if sliders feel laggy	Cap paths, cache by input hash, seed for repeatability
Database or Spacetime setup eats the first hours	Delays every other chunk	Assign one person to spike it immediately
Users read probabilities as guarantees	Misleading confidence	Ranges everywhere, disclaimer near every probability

Open questions

- [ ] Spacetime: enter as a stretch (live push of Copilot events to the dashboard) or skip? It needs one owner who spikes it in the first hours.
- [ ] Gemini: confirm current model names, free-tier limits, and track rules at the Google Cloud sponsor table.
- [ ] Backend language: TypeScript for one codebase, or Python for the simulator?
- [ ] 8090.AI: enter only if the Legacy Data Adapter fits in about 3 hours after P0 is done; otherwise skip.
- [ ] Exact schema and where the P1/P2 line falls once the first build hours are spent.

Next steps

- [ ] Team reads this and confirms the P0 cut.
- [ ] Claim chunks from section 12.3.
- [ ] Finalize the data model and simulator design.
- [ ] Set up hosting, database, and the synthetic profile.
- [ ] Write the 2-minute demo script around the rental and checkout scenarios.